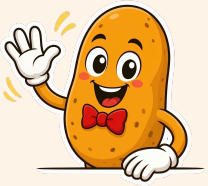
Mikhuna

Documentación del proyecto

Planificación de comidas según quién eres, con IA que elige entre recetas reales, dónde comprar cada ingrediente y delivery.







Equipo

Hubert Hilario Gomez · Abner Gonzales Alcarraz

Jesus Pezo Ramirez · Edward Pittman Medina

Generado el 21 de octubre de 2026 desde el repositorio

Contenido

1. Presentación README.md
2. Enfoque y diseño de la IA docs/ENFOQUE.md
3. Flujos de usuario docs/FLUJOS.md
4. Plan semanal con IA y reglas docs/IA.md
5. Arquitectura docs/ARQUITECTURA.md
6. Marca docs/BRANDING.md
7. Dataset de recetas docs/DATASET.md
8. Correr en localhost docs/LOCALHOST.md
9. Compilar el APK docs/COMPILACION_APK_ANDROID.md
10. Registro de cambios docs/CHANGELOG.md

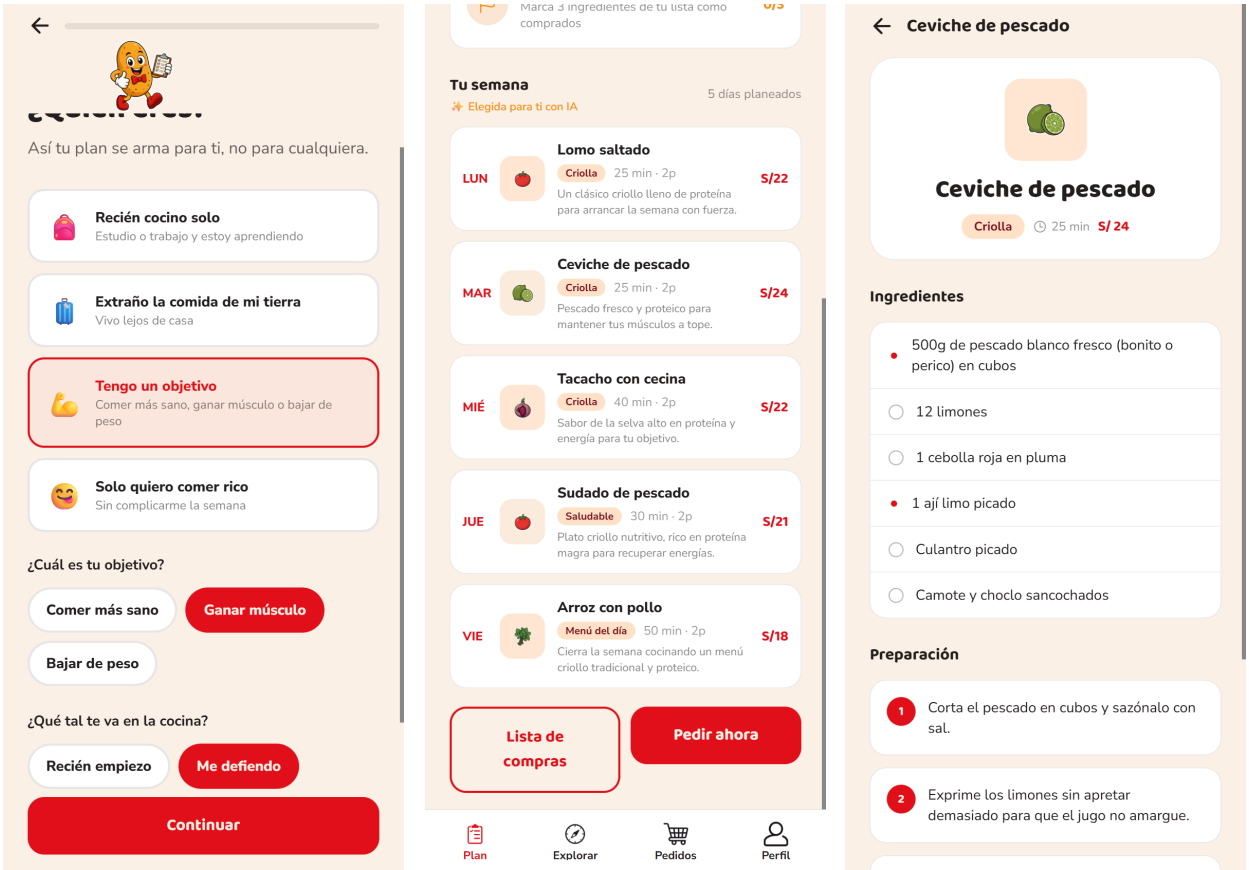
Mikhuna

La app para quien no sabe qué cocinar 🥚🌽🐭

Te armamos la semana según quién eres, te decimos dónde comprar cada ingrediente y, si no quieres cocinar, lo pides.

Mikhuna viene del quechua y significa **"comida"**.





¿Qué es Mikhuna?

Las apps de delivery parten de *"¿qué restaurante pido?"* y asumen que ya decidiste qué comer. **Mikhuna empieza un paso antes:** *"según quién soy y lo que puedo comer, ¿qué me conviene cocinar o pedir esta semana?"*

Le cuentas quién eres, qué se te antoja, para cuántos cocinas, cuánta plata tienes y qué hay en tu cocina. Mikhuna te arma un **plan de comidas concreto**, peruano o de otra cocina, te explica **por qué cada plato te sirve**, compara el precio de cada ingrediente entre bodegas, mercados y supermercados y, si un día no tienes ganas de cocinar, **pides al toque**.

Para quién es



El que recién cocina solo

Estudiantes y gente que recién se independiza: recetas fáciles, sin pasos imposibles.



El que extraña su tierra

Foráneos lejos de casa: comida de la costa, la sierra o la selva, como la de siempre.



El que tiene un objetivo

Fitness, intolerancias o dietas: "cualquier receta" no le sirve.

Lo que puedes hacer

"¿Quién eres?"	El plan empieza por ti: tu perfil, la comida que extrañas, tu objetivo y tu nivel en la cocina.
Plan semanal	Wizard de 8 pasos: quién eres, tienda, personas, días, presupuesto, antojos, restricciones y cocina.
Elegido con IA	Gemini elige entre recetas reales y te dice por qué cada una te va. Nunca inventa platos ni precios.
Respeta lo que comes	Vegano, vegetariano, pescetariano, sin gluten o sin lactosa: lo que no puedes comer nunca aparece.
Con lo que tienes	Si no tienes horno o licuadora, no te toca una receta que los necesite.
Billetera	Cuánto te queda de presupuesto, con una barra que pasa de verde a rojo.
Lista de compras	Compara el precio de cada ingrediente entre tiendas o márcalo como comprado.
Explorar	Busca negocios o productos ("pollo" encuentra el mercado que lo vende) y filtra por categoría.
Delivery	Carrito → checkout → seguimiento en vivo con el cuy → calificación.
Tus pedidos	Historial, lo que está en camino y "Volver a pedir" con los precios de hoy.
Cuenta con código	Registro y "Olvidé mi contraseña" con un código que llega a tu correo.

En la nube Tu plan y tus pedidos se guardan en Supabase: cambias de celular y siguen ahí.



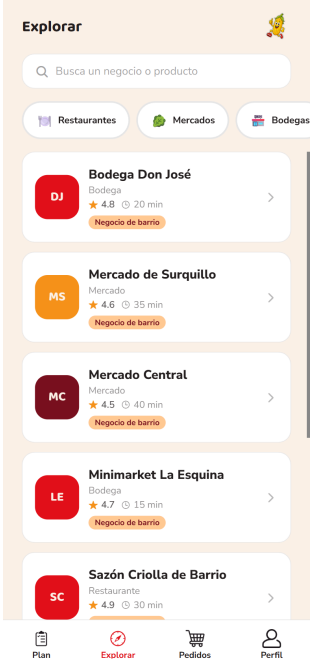
¿Qué tienes en tu cocina?



Papi Papa armando tu semana



Tu billetera y tu misión



Negocios del barrio

Cómo funciona la IA

Usamos un **modo híbrido: reglas + IA**. Lo que no se negocia lo decide el código; lo que es criterio, la IA.

```
Tus respuestas del wizard
        │
        ▼
1. Reglas (sin IA)        sacan toda receta que choque con tus restricciones o tu cocina
        │                 y ordenan el resto según quién eres → hasta 20 candidatas
        ▼
2. Gemini                 elige una receta por día SOLO de esa lista (no puede inventar)
        │                 y escribe en una frase por qué te sirve
        ▼
3. La app                 pone precios, cantidades y dónde comprar, desde el catálogo real
        │
        └─ ¿Gemini tarda, falla o no hay internet? Las mismas reglas arman el plan igual.
```

- **Nunca se salta una restricción:** `npm test` prueba todas las combinaciones de restricciones, cocinas y perfiles.
- **Rápido:** le pedimos el plan a Gemini de tres formas a la vez y usamos la primera que llega (~5 s).
- **Seguro:** la función exige una sesión real y solo acepta valores que la app puede mandar.

Detalle en docs/IA.md y por qué lo diseñamos así en docs/ENFOQUE.md.

🚀 Arráncalo

```
npm install
npx expo start --web      # o sin --web para Expo Go / emulador
npm test                  # pruebas de las reglas del plan
```

Crea un `.env.local` (no se sube a git):

```
EXPO_PUBLIC_SUPABASE_URL=...
EXPO_PUBLIC_SUPABASE_ANON_KEY=...

# opcional: entra solo en desarrollo con una cuenta de pruebas (no queda en el APK)
EXPO_PUBLIC_DEV_EMAIL=...
EXPO_PUBLIC_DEV_PASSWORD=...
```

¿Primera vez con la base de datos o la IA?

```
npx supabase link --project-ref lvkweyiwzkqtojlylinc
npx supabase db push                                  # tablas, RLS y límites
npx supabase functions deploy generar-plan --no-verify-jwt  # la función de IA
```

> 💡 Verlo con tamaño de celular en el navegador: docs/LOCALHOST.md. Sacar el APK de Android: docs/COMPILACION_APK_ANDROID.md. Correos con código: docs/FLUJOS.md.

🎨 La marca

Color	Hex	Para qué
chili	`#E5131B`	Acciones, precios, botones
mango	`#F7941D`	Estados, badges, íconos
peach	`#FFC993`	Chips y resaltados suaves
cream	`#FDF3E7`	Fondo de toda la app
maroon	`#7A1220`	Texto sobre fondos claros
ink	`#241F21`	Texto y contornos

Títulos en **Baloo 2**, textos en **Nunito Sans**. Las mascotas: **Papi Papa** arma tu plan (con versiones de costa, sierra y selva), **Choclito** te lleva a explorar y el **Cuy Delivery** te trae el pedido. Todo en docs/BRANDING.md.

🧰 Stack

- **Expo SDK 57** (bare workflow, `android/` en el repo) + **expo-router** con grupos de rutas

- **NativeWind v4** — Tailwind para React Native
- **Supabase** — auth con código por correo (SMTP con Resend), tablas `planes` y `pedidos` con RLS y la Edge Function `generar-plan` (Deno)
- **Gemini** — elige el plan entre recetas reales, con reglas de respaldo
- **React Context** para el estado global y **TypeScript** de punta a punta

🗂️ Cómo está armado

```
app/                    pantallas (expo-router, agrupadas por flujo)
components/             piezas reutilizables (WizardScreen, BrandButton, CampoTexto, KitchenScene)
context/                estado global (Auth, PlanWizard, Cart, Orders)
lib/                    catálogo, cliente de Supabase, armado del plan (ia.ts), tema
supabase/functions/
  _shared/              recetas y reglas del plan, compartidas por la app y la función
  generar-plan/         Edge Function: reglas + Gemini
supabase/migrations/    tablas, RLS y límites
supabase/templates/     correos de registro y recuperación
scripts/                dataset de recetas (Python), pruebas, plantillas de correo
src/assets/             mascotas, íconos, fondos de cocina, fotos de platos
docs/                   documentación
```

📚 Documentación

Doc	De qué va
ENFOQUE	Qué promete Mikhuna, qué falta y por qué la IA funciona así
FLUJOS	Onboarding, cuenta, plan semanal, compras, delivery, pedidos y perfil
IA	El plan con reglas + Gemini: cómo funciona, cómo probarlo y cómo agregar recetas
ARQUITECTURA	Carpetas, rutas, contexts y datos
BRANDING	Colores, tipografía, mascotas y tono de voz
DATASET	Dataset de recetas: fuentes, limpieza y cantidades
LOCALHOST	Correr en web, verlo como celular y problemas conocidos
COMPILACION_APK_ANDROID	Generar el APK
CHANGELOG	Qué cambió en cada commit y cómo regresarlo

🤝 Equipo



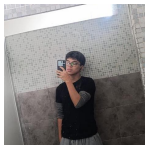
Hubert Hilario Gomez



Abner Gonzales Alcarraz



Jesus Pezo Ramirez



Edward Pittman Medina

Hecho en Perú con harto ají 🌶️

CAPÍTULO 2 · DOCS/ENFOQUE.MD

Enfoque de Mikhuna y cómo debe funcionar la IA

Revisión de la presentación del equipo (21 oct 2026) contra lo que hace hoy la app, y la propuesta de cómo cerrar la diferencia. Para el detalle técnico de lo que ya existe: FLUJOS.md, IA.md, DATASET.md.

Lo que promete la presentación

> Las apps de delivery parten de "¿qué restaurante pido?". Mikhuna empieza un paso antes: **"según quién soy o qué puedo comer, ¿qué me conviene cocinar o pedir esta semana?"**

1. **Para quién:** quien no sabe qué cocinar. Tres perfiles concretos: el **estudiante** que recién cocina solo, el **foráneo** lejos de su comida de siempre, y quien tiene una **restricción u objetivo** (fitness, intolerancias, dietas) para quien "cualquier receta" no sirve.
2. **Qué entrega:** un **plan de comidas concreto** para la semana, **peruano o de otra cocina**, según lo que el usuario elija.
3. **Y sobre ese plan: dónde conviene comprar cada ingrediente**, para cocinarlo o pedirlo.
4. **Con qué:** React Native, Expo, TypeScript, Supabase y **Gemini**.

Lo que hace la app hoy

Promesa	Hoy	Brecha
"Según quién eres"	El wizard pregunta personas, días, presupuesto, antojos, restricciones y electrodomésticos.	No pregunta **quién es** (estudiante, foráneo, objetivo) ni su **nivel en la cocina**.
Plan que respeta lo que puedes comer	`generarPlanSemana()` reparte **8 recetas fijas** según los antojos.	Ignora restricciones, presupuesto, personas y electrodomésticos. Un vegano puede recibir lomo saltado.
"Peruano o de otra cocina"	Las 8 recetas son peruanas.	No hay otras cocinas ni la comida "de tu tierra" para el foráneo.
IA (Gemini)	La función `generar-plan` existe, está desplegada y protegida, pero **el wizard no la usa**.	La pantalla "Generando" es solo una animación.
Dónde comprar cada ingrediente	Funciona bien, pero solo para **8 ingredientes** del catálogo.	Una receta real tiene 10–15 ingredientes; casi ninguno está en el catálogo.
Cocinar o pedir	Funciona: lista de compras, comparar tiendas, delivery, seguimiento.	—

Diagnóstico: la parte de "qué hago con el plan" (comprar, pedir, seguir el pedido) está bien resuelta. Lo que flojea es justo lo que diferencia a Mikhuna: **el plan en sí**. Hoy no depende de quién eres, y eso es lo primero que va a notar alguien que pruebe la app o un jurado.

Cómo debe funcionar la IA

La regla de fondo: **la IA no inventa recetas ni precios; elige entre recetas reales y explica por qué te sirven.** Un modelo que inventa platos puede saltarse una alergia o proponer algo que no existe; uno que elige de una lista cerrada, no.

```
Wizard (quién eres + qué puedes comer)
        │
        ▼
1. Filtro duro (SQL en Supabase, sin IA)      ← dataset de ~28 800 recetas reales
   restricciones, electrodomésticos, dificultad, cocina/país
        │  ~40 recetas candidatas
        ▼
2. Gemini elige y explica (Edge Function)
   recibe el perfil + las candidatas (id, nombre, ingredientes)
   devuelve 1 id por día + "por qué te va" en una frase
   el esquema de respuesta solo admite ids de la lista → no puede inventar
        │
        ▼
3. La app arma todo lo demás (sin IA)
   precios contra el catálogo de tiendas, lista de compras sumando cantidades,
   porciones según personas, dónde conviene comprar
        │
        └─ si Gemini falla o no hay cuota: el mismo paso 2 con reglas (puntaje)
```

- **Paso 1, lo que no se negocia, lo decide código:** si eres vegano o celíaco, el filtro saca esas recetas antes de que la IA las vea. El dataset ya trae `apta_vegana` / `apta_vegetariana` calculadas desde los ingredientes, no desde la etiqueta de la fuente.
- **Paso 2, lo que es criterio, lo decide la IA:** "soy estudiante y es mi primera vez", "quiero subir masa muscular", "extraño la comida de Arequipa", "algo rapidito entre semana". Ahí Gemini aporta lo que unas reglas no pueden: entender la intención y variar la semana (no tres sopas seguidas).
- **Paso 3, los números, los calcula la app:** precios, cantidades y presupuesto siempre salen del catálogo, nunca de la IA (ya funciona así en `lib/ia.ts`).
- **"IA local":** no conviene un modelo dentro del celular (cientos de MB, lento, gasta batería). Lo "local" es el filtro y el respaldo por reglas: el plan se arma aunque Gemini esté caído.
- **Costo:** mandar 40 candidatas resumidas son pocos tokens; cabe holgado en la capa gratis de Gemini. Falta un límite de llamadas por usuario (ver IA.md).

¿Qué quiere decir "modo híbrido"?

En el grupo se habló de que la IA funcione en "modo híbrido". La palabra se usa para tres cosas distintas; esto es lo que significa cada una para Mikhuna:

Significado	Qué es	¿Lo usamos?
Reglas + IA	El código filtra lo que no se negocia y calcula los números; Gemini elige y explica; si Gemini falla, las reglas eligen solas.	**Sí, es la propuesta de arriba.**

Significado	Qué es	¿Lo usamos?
Búsqueda híbrida	Buscar recetas por palabras ("pollo", "Arequipa") y por significado con vectores (pgvector: "algo ligero para la noche").	**Sí, más adelante**, para encontrar las candidatas del paso 1 en el dataset.
Celular + nube	Un modelo chico dentro del teléfono (Gemini Nano) para lo simple y Gemini en la nube para lo difícil.	**No.** Hoy solo corre en algunos celulares de gama alta.

Cuando digamos "modo híbrido" en el proyecto, nos referimos al primero.

Qué construir, en orden

Puntaje = impacto × confianza ÷ esfuerzo (1–5 cada uno).

#	Qué	Por qué	Puntaje
1	**Paso "¿Quién eres?"** en el wizard: estudiante que recién cocina / lejos de casa (qué cocina extraña) / tengo un objetivo (fit, bajar, subir) / solo quiero comer rico. Más nivel en la cocina.	Es la promesa central de la presentación y hoy no existe. Barato: una pantalla y dos campos.	5×5÷1 = **25**
2	**Plan local que respete restricciones, presupuesto y electrodomésticos** (aunque sea con las recetas actuales).	Hoy un vegano recibe carne. Arreglarlo no depende de nada.	5×5÷2 = **12.5**
3	**Cargar un subconjunto del dataset a Supabase**: recetas con ingredientes bien separados, peruanas primero y algunas de otras cocinas (unas 2 000–5 000), con dificultad estimada y electrodomésticos detectados en los pasos.	Sin recetas reales no hay plan "según quién eres". **Necesita los CSV** (no están en esta compu).	5×4÷3 ≈ **6.7**
4	`generar-plan` **v2**: filtro SQL → Gemini elige por id con "por qué te va" → respaldo por reglas. Conectarla al wizard.	Es la IA de la presentación, hecha sin que pueda inventar.	5×4÷3 ≈ **6.7**
5	**Ampliar el catálogo de ingredientes** con los ~50 más frecuentes del dataset, con precios de referencia marcados como demo.	"Dónde comprar cada ingrediente" solo convence si la lista sale completa.	4×4÷3 ≈ **5.3**

Estado (21 oct 2026): el 1, el 2 y el 4 ya están hechos con las 20 recetas del catálogo (`supabase/functions/_shared/recetas.ts`): Gemini elige entre ellas y las reglas son el respaldo. Falta el 3 (cargar el dataset, que necesita los CSV) para pasar de 20 recetas a miles, y el 5. Cuando el dataset esté en Supabase, solo cambia de dónde salen las candidatas; el resto del diseño queda igual. Ver IA.md.

Lo que no vamos a hacer (por ahora)

- **Un modelo de IA dentro del celular.**
- **Dejar que la IA invente recetas, cantidades o precios.**
- **Prometer seguridad para alergias.** El filtro ayuda, pero los datos vienen de sitios de recetas: la app debe decir que revise los ingredientes si tiene una alergia seria.
- **Competir con Rappi o PedidosYa en "pedir algo ya".** El Flujo B sirve, pero la diferencia de Mikhuna es el plan; ahí va el esfuerzo.

- **Pagos reales.**

Cómo sabremos que funciona

- **Plan aceptado:** de cada plan generado, cuántos el usuario deja sin cambiar recetas (meta inicial: más de la mitad).
- **Receta abierta:** cuántos platos del plan se abren para ver la preparación.
- **Del plan a la compra:** cuántos planes terminan en lista de compras marcada o pedido de ingredientes.
- **Cero violaciones:** ningún plan con una receta que choque con las restricciones del usuario (se puede probar automáticamente).

Riesgos

- **Licencia del dataset.** Las recetas vienen de sitios comerciales; vale para el curso y el prototipo, no para publicar la app sin revisarlo (ver DATASET.md).
- **Calidad de los datos.** Cerca de la mitad de los ingredientes no tiene cantidad confiable; por eso conviene cargar solo el subconjunto con ingredientes bien separados.
- **Cuota de Gemini.** Sin límite por usuario, alguien con sesión puede agotarla; el respaldo por reglas evita que la app se quede sin plan.

CAPÍTULO 3 · DOCS/FLUJOS.MD

Flujos de usuario

1. Onboarding y autenticación

1. `app/index.tsx` revisa si hay sesión de Supabase (`AuthContext`). - Sin sesión → `(auth)/login` (o `(auth)/registro` desde "Regístrate"). - **Registro** → al crear la cuenta pide el código que llega al correo (`verifyOtp` tipo `email`) y entra directo a la app; se puede reenviar cada 60 s. Si alguien intenta entrar con una cuenta sin confirmar, el login le ofrece "Confirmar con un código", que abre ese mismo paso. Usa la plantilla `supabase/templates/confirmar-registro.html` (más abajo, cómo subirla). - Con sesión y distrito ya elegido → `(tabs)` directo. - Con sesión y sin distrito (primera vez) → `(onboarding)/ubicacion`. - **"¿Olvidaste tu contraseña?"** en el login → `(auth)/recuperar`: el usuario escribe su correo, Supabase le manda un código (`resetPasswordForEmail`), y con el código y la contraseña nueva (`verifyOtp` tipo `recovery` + `updateUser`) entra directo a la app. Se usa código y no enlace porque la app no tiene deep links configurados.

   Los correos de registro y recuperación tienen diseño propio (logo, Papi Papa y el código grande): sus plantillas están en `supabase/templates/` y se suben a Supabase con `powershell -ExecutionPolicy Bypass -File scripts/supabase/aplicar-plantillas-correo.ps1` (usa la sesión de `npx supabase login`). Si se editan en el dashboard en vez de aquí, se pierden al volver a correr el script.

   Requiere SMTP propio. Con el servidor de correo por defecto, Supabase (plan gratuito) no deja cambiar las plantillas ("Email template modification is not available for free tier projects using the default email provider"): el correo llega con un enlace y no con código, y además solo se envía a los miembros del equipo. Con eso el registro funciona igual (abrir el enlace confirma la cuenta), pero "Olvidé mi contraseña" no.

   Configuración actual: Resend con el dominio `abnergonzales.dev` (verificado en Resend; sus registros DNS están en Vercel). Los correos salen de `no-reply@abnergonzales.dev` como "Mikhuna". 1. En `.env.local`: `RESEND_API_KEY=re_...` (API key con permiso de envío) y `SMTP_REMITENTE=no-reply@abnergonzales.dev`. 2. Correr `scripts/supabase/aplicar-plantillas-correo.ps1`: configura `smtp.resend.com` (puerto `465`, usuario `resend`) y sube las plantillas en el mismo paso.

   El script también acepta Gmail (`SMTP_GMAIL_USER` y `SMTP_GMAIL_APP_PASSWORD`, una contraseña de aplicación de https://myaccount.google.com/apppasswords) si alguna vez se quiere volver a eso. El plan gratis de Resend da 100 correos al día; Supabase además limita los correos por hora (Authentication → Rate Limits). El código que llega es de 8 dígitos y vence en 1 hora. 2. `(onboarding)/ubicacion` — el usuario elige su distrito (o usa su ubicación actual), para saber qué negocios mostrarle cerca. 3. `(onboarding)/modo` — "¿Qué quieres hacer hoy?", la bifurcación central de la app: - **"Quiero planear mi semana"** → entra al Flujo A (`(plan)/negocio`). - **"Quiero pedir algo ahora"** → entra al Flujo B (`(tabs)/explorar`).

Desde el tab "Plan" siempre se puede volver a armar un plan nuevo, y desde cualquier lado se puede ir a "Explorar" para el Flujo B — no es una elección de una sola vez.

2. Flujo A — Plan semanal

Wizard de 8 pasos, todos usando `WizardScreen` (barra de progreso + Papi Papa caminando), guardando cada respuesta en `PlanWizardContext`:

1. **`quien-eres`** — ¿quién eres? Recién cocino solo / extraño la comida de mi tierra (costa, sierra o selva) / tengo un objetivo (más sano, ganar músculo, bajar de peso) / solo quiero comer rico, y qué tal le va en la cocina.
2. **`negocio`** — ¿dónde haces tus compras? (negocio de referencia para el distrito).
3. **`hogar`** — ¿para cuántos cocinas? (número de personas).
4. **`dias`** — ¿qué días cocinarás esta semana?
5. **`presupuesto`** — presupuesto semanal (slider S/40–S/600).
6. **`antojo`** — hasta 3 antojos (criolla, rápida, saludable, menú del día, vegetariana, fit/proteica, económica, chifa).
7. **`restricciones`** — alergias/restricciones alimentarias.
8. **`electrodomesticos`** — qué tiene en su cocina, con la escena interactiva `KitchenScene` (niveles básica/promedio/avanzada).

Al terminar, **`generando`** arma el plan de verdad mientras Papi Papa cocina: le pide a la Edge Function que Gemini elija entre las recetas que cumplen sus restricciones y su cocina, y si eso falla lo arma con las mismas reglas en el celular. El plan se guarda (`planSemana`) y no cambia hasta rehacer el wizard. Detalle en IA.md.

Home del plan (`(tabs)/index.tsx`)

Con el plan ya armado, la pantalla "Plan" muestra:

- Saludo + racha (días planeados) + comidas listas.
- Tarjeta de **billetera**: disponible para gastar, gastado vs. presupuesto, con una barra de color dinámico (verde/naranja/rojo según qué tan cerca está del límite).
- **Misión del día**: marcar 3 ingredientes de la lista como comprados — dato real de `PlanWizardContext`, no inventado.
- **Tu semana**: un plato por día, con una frase de por qué le sirve ("✨ Elegida para ti con IA" si la armó Gemini). Si ninguna receta cumple a la vez sus restricciones y su cocina, se le dice y se le ofrece ajustar el plan. Tocar un plato abre `(plan)/receta/[id]` con ingredientes y pasos de preparación; los ingredientes que también existen en `INGREDIENTES` se marcan como "Ya comprado" si ya se marcaron en la lista de compras.

Lista de compras

Desde el home, "Lista de compras" abre **`(plan)/lista-compras`** — un modal flotante (no pantalla completa) con dos caminos:

- **"Pedir por delivery"** → `(plan)/comprar-delivery`: el usuario elige una tienda principal; lo que esa tienda no vende se reparte automáticamente entre las demás tiendas que sí lo tienen, agrupado por negocio.

- **"Voy a comprarlo yo"** → `(plan)/comprar-yo-mismo`: checklist plano de todos los ingredientes, cada uno se puede marcar como comprado. Tocar un ingrediente abre `(plan)/ingrediente/[id]`, que compara el precio en cada tienda que lo vende y deja asignarlo a una (sin salir de la pantalla, para poder seguir comparando el resto).

Las asignaciones (`preferenciasCompra`) alimentan `(plan)/carrito-ingredientes`: un carrito agrupado por tienda, con buscador, donde cada grupo tiene su propio botón de "Pedir de [tienda]".

3. Flujo B — Delivery inmediato

`(tabs)/explorar` muestra negocios y categorías cercanas. Entrar a un negocio (`(delivery)/negocio/[id]`) muestra su catálogo de platos/productos, se arma el carrito (`CartContext`) y se pasa por `(delivery)/carrito` → `checkout` → `seguimiento` → `calificar` al terminar.

Este flujo comparte `CartContext` y las pantallas de `(delivery)` con el Flujo A: cuando el usuario pide delivery de ingredientes desde el plan semanal, termina en el mismo carrito/checkout/seguimiento que un pedido armado desde Explorar.

4. Perfil y pedidos

- `(tabs)/pedidos` — arriba los pedidos en curso (chip de estado y barra de pasos, abren el seguimiento) y debajo los entregados, con fecha, productos, total, estrellas o "Calificar" y "Volver a pedir" (rellena el carrito). La pestaña muestra un contador cuando hay pedidos en curso.
- `(tabs)/perfil` — nombre, correo, personas, presupuesto y número de pedidos; accesos para cambiar de distrito, rehacer el plan y ver pedidos; restricciones elegidas; y "Cerrar sesión" (pide un segundo toque). Cerrar sesión vacía el carrito pero el plan queda guardado para la próxima vez.

CAPÍTULO 4 · DOCS/IA.MD

Plan semanal con IA (Gemini) y reglas

Estado: **conectado al wizard.** Al terminar el wizard, la pantalla "Generando" arma el plan: primero con Gemini y, si eso falla, con las reglas en el mismo celular. El plan queda guardado en `planSemana` (con `planFuente: "ia"` o `"reglas"`) y "Tu semana" lo muestra con una frase de por qué cada plato le sirve. Por qué se diseñó así: ENFOQUE.md.

Cómo funciona

```
Wizard ("¿Quién eres?", días, presupuesto, antojos, restricciones, electrodomésticos)
   │
   ▼  lib/ia.ts → armarPlan()
App ──invoke("generar-plan")──► Edge Function (Deno)
                                  1. valida el perfil (solo valores conocidos)
                                  2. _shared/plan.ts: filtra restricciones y cocina,
                                     ordena por perfil → hasta 20 candidatas
                                  3. Gemini elige 1 id por día + "por qué te va"
                                     (3 variantes a la vez, gana la primera; máx. 18 s)
                                  4. completa lo que falte con las reglas
   ◄── { platos: [{ dia, recetaId, razon }], fuente: "ia" | "reglas" }
   │
   └─ si no responde en 22 s, falla o no calza: armarPlanLocal() en el celular
```

- **La IA no inventa recetas ni precios.** Solo elige entre las recetas de `supabase/functions/_shared/recetas.ts`, las mismas que muestra la app. Precios, cantidades y la lista de compras salen siempre del catálogo.
- **Lo que no se negocia lo decide código.** `_shared/plan.ts` saca antes de llegar a Gemini toda receta que choque con las restricciones (vegano, vegetariano, pescetariano, sin gluten, sin lactosa) o que pida un electrodoméstico que la persona no tiene. La app vuelve a revisar la respuesta antes de usarla.
- **Lo que es criterio lo decide la IA:** quién es (estudiante, foráneo, con un objetivo), su nivel en la cocina, sus antojos y su presupuesto, y que la semana varíe.
- **Las reglas son el respaldo y también la base:** sin internet, sin cuota o sin `GEMINI_API_KEY`, el plan sale igual. `armarPlanLocal()` da puntos por perfil, castiga repetir y nunca se salta una restricción: si nada cumple, devuelve un plan vacío y la app le dice a la persona qué ajustar.
- **Un solo código para la app y la función.** `supabase/functions/_shared/` lo importan `lib/catalogo.ts`, `lib/ia.ts` y la Edge Function. Por eso importa con extensión `.ts` (Deno lo exige) y el `tsconfig.json` de la app tiene `allowImportingTsExtensions`.

Configuración

- La API key de Gemini vive como **secreto del proyecto de Supabase** (`GEMINI_API_KEY`), nunca en el cliente ni en el repo: `npx supabase secrets set GEMINI_API_KEY=...`.
- El modelo se cambia con el secreto `GEMINI_MODEL` (por defecto `gemini-2.5-flash`). Las cuentas nuevas de Google AI Studio ya no tienen acceso a `gemini-2.5-flash` y responden `404`; en ese caso: `npx supabase secrets set GEMINI_MODEL=gemini-3.8-flash`.

- Desplegar (después de cambiar la función **o** cualquier archivo de `_shared/`): `bash npx supabase functions deploy generar-plan --no-verify-jwt` Si la función desplegada es más vieja que la app, la respuesta no calza y la app usa las reglas: el plan sale igual, pero sin "✨ Elegida para ti con IA".

Probarla

Las reglas, sin internet ni Supabase: `npm test`. Prueba, entre otras cosas, todas las combinaciones de restricciones con varios tipos de cocina y perfiles: ningún plan puede traer algo que la persona no pueda comer o preparar.

La función, que exige la sesión de un usuario real, así que hace falta su `access_token`. El más fácil de sacar es el de la app web en modo desarrollo: en la consola del navegador, `JSON.parse(localStorage.getItem(Object.keys(localStorage).find(k => k.includes("auth-token")))).access_token`.

```
curl -s -X POST "https://lvkweyiwzkqtojlylinc.supabase.co/functions/v1/generar-plan" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <access_token>" \
  -d '{"dias":["Lunes","Martes"],"antojos":["criolla"],"restricciones":["vegano"],"electrodomesticos":["cocina"],
      "presupuesto":150,"personas":2,"perfil":"foraneo","region":"sierra","objetivo":null,"nivelCocina":"principi
```

Sin token responde `401`; con valores que la app no manda (un día mal escrito, un antojo inventado, más de 12 personas) responde `400`. Cuando responde con las reglas trae `motivo` (`gemini_tardo`, `gemini_429`, `gemini_sin_texto_MAX_TOKENS`...). El detalle queda en los logs del dashboard: https://supabase.com/dashboard/project/lvkweyiwzkqtojlylinc/functions/generar-plan/logs.

Seguridad y costo

- **Quién puede llamarla:** se despliega con `--no-verify-jwt`, pero la propia función pide un usuario con sesión (`auth.getUser()` con el header `Authorization`) y responde `401` si no lo hay. El anon key solo no basta: va dentro del APK y cualquiera podría gastarse la cuota de Gemini con él.
- **Entrada cerrada:** solo acepta días, antojos, restricciones, electrodomésticos, perfil, región, objetivo y nivel que existen en la app, un presupuesto entre 0 y 10 000 y de 1 a 12 personas. En el prompt no entra texto libre del usuario, así que no se le pueden meter instrucciones.
- **Velocidad:** `gemini-3.8-flash` tardaba más de 18 s en elegir. Por eso la función le pide el plan de tres formas a la vez (con esquema, sin esquema y con el modelo liviano `gemini-flash-lite-latest`, que se cambia con el secreto `GEMINI_MODEL_RAPIDO`) y usa la primera que dé un plan válido; las otras se cancelan. Si ninguna llega en 18 s, responde con las reglas. La respuesta trae `variante` y `ms` (o `motivo` con lo que le pasó a cada una). Medido el 21 oct 2026: ganó el modelo liviano en 4.7 s.
- **Costo:** cada plan puede gastar hasta 3 llamadas (cada modelo tiene su propia cuota). El prompt lleva como mucho 20 recetas resumidas.
- **Falta todavía:** un límite de llamadas por usuario (por ejemplo, una tabla `uso_ia` con un máximo por hora) y cachear el plan por perfil para no pedirle a Gemini dos veces lo mismo.

Agregar recetas

Se agregan en `_shared/recetas.ts` con todos sus campos (dieta, gluten, lactosa, electrodomésticos, dificultad, región, etiquetas) y `npm test` revisa que sean coherentes. Si es un antojo, restricción o electrodoméstico nuevo, también va en `ANTOJOS_IDS` / `RESTRICCIONES_IDS` / `ELECTRODOMESTICOS_IDS` de `_shared/plan.ts` y en `lib/catalogo.ts`. Después, volver a desplegar la función. Cuando el dataset se cargue a Supabase, esta lista se reemplaza por esa tabla (DATASET.md).

CAPÍTULO 5 · DOCS/ARQUITECTURA.MD

Arquitectura

Ruteo (expo-router)

Todas las pantallas viven en `app/`, agrupadas por paréntesis (route groups) que no afectan la URL pero organizan el flujo:

- `(auth)` — login, registro y recuperar contraseña.
- `(onboarding)` — ubicación del usuario (`ubicacion.tsx`, con `mapa.tsx` para confirmarla) y elección de modo ("quiero planear mi semana" vs "quiero pedir algo ahora", `modo.tsx`).
- `(plan)` — Flujo A completo: wizard de plan semanal de 8 pasos, generación del plan (`generando.tsx`), lista de compras y detalle de ingrediente y de receta. Ver FLUJOS.md.
- `(tabs)` — las 4 pestañas principales una vez el usuario ya tiene cuenta y ubicación: Plan, Explorar, Pedidos, Perfil.
- `(delivery)` — carrito, checkout, seguimiento, calificación y menú de un negocio (`negocio/[id]`). Lo usan tanto el Flujo A como el Flujo B.

Orden de los pasos del wizard (cada pantalla declara su `paso` para la barra de progreso): `quien-eres` (1), `negocio` (2), `hogar` (3), `dias` (4), `presupuesto` (5), `antojo` (6), `restricciones` (7), `electrodomesticos` (8).

`app/_layout.tsx` monta los providers y un `GuardiaSesion`: si la sesión se pierde fuera de `(auth)` (venció, se revocó o se cerró en otro lado), manda al login.

`app/index.tsx` es el punto de entrada: sin sesión va a `(auth)`; con sesión espera a que cargue el plan guardado y va a `(tabs)` si ya hay distrito elegido, o a `(onboarding)` si es la primera vez.

Estado global (`context/`)

Cuatro contexts de React, sin librería externa de estado:

- `AuthContext` — sesión de Supabase. En `__DEV__` auto-loguea con la cuenta de pruebas de `.env.local` (`EXPO_PUBLIC_DEV_EMAIL` / `EXPO_PUBLIC_DEV_PASSWORD`) para iterar más rápido; sin esas variables muestra el login normal. No corre en producción: esa rama se quita del build de release. `loading` no termina hasta que se resuelve ese intento. Expone `nombre` (el del registro o, si falta, lo que va antes de la @ del correo) y traduce al español los errores de Supabase.
- `PlanWizardContext` — todas las respuestas del wizard de Flujo A: perfil ("¿Quién eres?"), distrito, negocio, días, presupuesto, antojos, restricciones, electrodomésticos, región (la comida que extraña, si es foráneo), objetivo, nivel de cocina, ingredientes marcados como comprados (`comprados`) y en qué tienda quiere comprar cada uno (`preferenciasCompra`). También guarda el plan ya elegido: `planSemana` (un plato por día) y `planFuente` (`"ia"` o `"reglas"`). Se guarda para que no cambie cada vez que se abre la app. Un plan hecho antes de que existiera `planSemana` tiene `planFuente: null`, y `platosDeLaSemana()` lo arma con las reglas al abrirlo. Se guarda en la tabla `planes` de Supabase (un plan por usuario, en `jsonb`) con un upsert 800 ms después del último cambio, y se

mantiene una copia en AsyncStorage (`mikhuna:plan:<userId>`) para abrir rápido y sin internet. Si no se pudo leer del servidor, los cambios quedan solo en el celular y no se suben, para que un plan viejo no pise uno más nuevo hecho en otro dispositivo. `listo` indica que ya se leyó.

- `CartContext` — carrito de delivery, agnóstico de si el pedido viene de Flujo A (ingredientes) o Flujo B (platos de un negocio): cada ítem es autocontenido (`{ id, nombre, precio, cantidad }`), sin necesitar volver a buscar en un catálogo. `llenarCon(negocioId, items)` reemplaza el carrito por los ingredientes de una tienda ("Pedir de X" en la lista de compras). El carrito se vacía al cambiar de usuario.
- `OrdersContext` — historial de pedidos del usuario: checkout crea el pedido, `seguimiento` avanza su `estado` (`confirmado` → `preparando` → `en-camino` → `entregado`) y `calificar` guarda las estrellas. Vive en la tabla `pedidos` de Supabase: crear un pedido espera la respuesta del servidor (checkout muestra el error si falla); estado y calificación se guardan por detrás. Los pedidos que hubieran quedado en AsyncStorage de la versión anterior se suben una vez.

Capa de datos (`lib/`)

`lib/catalogo.ts` — catálogo y datos de prueba

Negocios e ingredientes son datos de prueba en el celular (todavía no hay backend de catálogo):

- `NEGOCIOS` — bodegas, mercados, restaurantes y supermercados, cada uno con categoría, rating, tiempo de entrega.
- `INGREDIENTES` — catálogo de ingredientes con **ofertas multi-tienda**: cada ingrediente tiene un array `ofertas: { negocioId, precio, unidad }[]`, y `ofertaElegida()` resuelve cuál usar (la preferencia guardada del usuario, o la más barata por defecto).
- `RECETAS` — re-exportadas desde `supabase/functions/_shared/recetas.ts` y con la foto agregada (`foto`, solo donde existe). Ver la sección siguiente.
- `TOTAL_PASOS` (= 8) — cantidad de pasos del wizard, para la barra de progreso de `WizardScreen`.
- `ANTOJOS`, `RESTRICCIONES`, `ELECTRODOMESTICOS`, `DISTRITOS`, `DIAS` — catálogos de opciones del wizard. `DIAS` viene de `_shared/plan.ts`.

`supabase/functions/_shared/` — recetas y reglas compartidas

La app y la Edge Function `generar-plan` usan los mismos dos archivos, así la IA solo puede elegir entre recetas que la app conoce:

- `recetas.ts` — las 20 recetas (`RECETAS`). Cada una tiene antojo principal y etiquetas, región, dieta (`carne`, `pescado`, `vegetariana`, `vegana`), sin gluten / sin lactosa, electrodomésticos que requiere, dificultad (1-3), tiempo, precio, `ingredientes: { texto, ingredienteId? }[]` y `pasos: string[]`. El `ingredienteId` es opcional y solo existe cuando ese ingrediente también está en `INGREDIENTES`, lo que permite cruzarlo con `comprados` y mostrar "Ya comprado" en la pantalla de receta.
- `plan.ts` — las reglas: ids válidos (`ANTOJOS_IDS`, `RESTRICCIONES_IDS`, `ELECTRODOMESTICOS_IDS`, `PERFILES`, `OBJETIVOS`, `NIVELES_COCINA`), `recetasPosibles()` (filtro duro: restricciones y electrodomésticos), `candidatas()`, `puntaje()` (quién eres, antojos, presupuesto y nivel de cocina dan puntos para ordenar), `razonDe()` y `armarPlanLocal()`.

Esos archivos no importan nada de React Native, así que corren también en Deno. Por eso se importan con extensión `.ts` (`allowImportingTsExtensions`).

`lib/ia.ts` — armar el plan semanal

- **`armarPlan(prefs)`** — intenta `generarPlanConIA()`: llama a la Edge Function `generar-plan` (espera máxima de 22 s) y revisa la respuesta día por día (que cada receta exista y esté permitida). Si algo falla, arma el plan con `armarPlanLocal()` y devuelve `fuente: "reglas"`. Si ninguna receta cumple las restricciones, devuelve `[]`. La usa `app/(plan)/generando.tsx`.
- **`preferenciasDe(data)`** — convierte las respuestas del wizard (`PlanData`) en `PreferenciasPlan`.
- **`platosDeLaSemana(data)`** — lo que muestra "Tu semana": el `planSemana` guardado, o las reglas al vuelo si el plan es de antes de `planFuente`.

Componentes compartidos (`components/`)

- **`WizardScreen`** — wrapper de las 8 pantallas del wizard de Flujo A: header con barra de progreso + mascota caminando sincronizada al paso actual, y contenido centrado verticalmente.
- **`BrandButton`** — botón primario/outline reutilizado en toda la app.
- **`CampoTexto`** — campo de texto de login, registro y recuperar contraseña.
- **`SafeAreaView`** — envoltorio de `react-native-safe-area-context`. En el celular es el original; en web (`SafeAreaView.web.tsx`) es un `View` plano para que las clases de Tailwind (`px-6`) no se pierdan. Todas las pantallas lo importan de acá.
- **`PhoneFrame`** (en `app/_layout.tsx`) — solo en web: limita la app a 430 px de ancho y la centra. Ver LOCALHOST.md.
- **`KitchenScene`** — la escena interactiva de "¿qué tienes en tu cocina?": fondo de cocina + electrodomésticos posicionados a mano (por nivel: básica/promedio/avanzada), tocables para marcar cuáles tiene el usuario.

Multi-store: por qué existe `preferenciasCompra`

El modelo evitó a propósito depender de "un solo negocio vende todo el plan": cada ingrediente puede venderse en varias tiendas a distinto precio, y el usuario decide (o deja que se elija automáticamente la más barata) dónde comprar cada uno. `preferenciasCompra: Record<ingredienteId, negocioId>` guarda esa decisión, y tanto la lista de compras como el carrito de delivery la leen para agrupar por tienda.

Backend (`supabase/`)

- **`functions/generar-plan/index.ts`** — Edge Function (Deno). Valida las preferencias que llegan, toma hasta 20 recetas candidatas con `_shared/plan.ts` y le pide a Gemini un plato por día, eligiendo solo entre esas recetas (plazo de 18 s; modelo configurable con `GEMINI_MODEL` y `GEMINI_MODEL_RAPIDO`). Si Gemini no responde o su respuesta no sirve, responde el plan de las reglas (`fuente: "reglas"`). La API key es un secreto del proyecto (`GEMINI_API_KEY`), nunca va al cliente. Ver IA.md.
- **`functions/_shared/`** — `plan.ts` y `recetas.ts`, compartidos con la app (ver arriba).

- **`migrations/`** — `20261021020000_planes_y_pedidos.sql` crea las tablas `planes` y `pedidos`, con RLS. `20261021030000_limites_de_tamano.sql` pone topes de tamaño a los campos de `pedidos` y a `planes.datos`.
- **`templates/`** — plantillas de correo de confirmar registro y recuperar contraseña. Se suben al proyecto con `scripts/supabase/aplicar-plantillas-correo.ps1`, que también puede configurar el SMTP (Resend o Gmail) con las variables de `.env.local`.
- `tsconfig.json` excluye `supabase/functions` porque corre en Deno, no en React Native.
- `supabase/.temp/` (estado local del CLI) está en `.gitignore`.

Pruebas (`scripts/pruebas/`)

- **`plan.test.ts`** — pruebas de las reglas del plan (`_shared/plan.ts`). Se corren con `npm test` (Node con `--experimental-strip-types`). `tsconfig.json` también excluye esta carpeta.

Scripts de datos (`scripts/dataset/`)

Pipeline en Python que arma la base de recetas en español a partir de CSV externos (dedupe, limpieza, ingredientes separados, cantidades normalizadas, tabla de peso por unidad) y un visor HTML local para revisarla. Los resultados van a `data/` (ignorado por git). Ver DATASET.md.

CAPÍTULO 6 · DOCS/BRANDING.MD

Branding

Nombre

Mikhuna — del quechua, "comida". La marca apuesta por lo peruano de verdad: platos criollos, mercados de barrio, bodegas, y un tono cercano y familiar, no corporativo.

Paleta de colores

Definida en `tailwind.config.js` y espejada en `lib/theme.ts`:

Token	Hex	Uso
`chili`	`#E5131B`	Color primario: botones, precios, acciones, barra de progreso
`mango`	`#F7941D`	Acento secundario: badges, íconos, estados intermedios
`peach`	`#FFC993`	Fondos suaves de chips/cápsulas, resaltados discretos
`cream`	`#FDF3E7`	Fondo base de toda la app
`maroon`	`#7A1220`	Texto sobre fondos color peach/chili claro, acentos oscuros
`ink`	`#241F21`	Texto principal, contornos

Regla de mano: `chili` es la acción, `mango` es el estado, `peach` / `cream` son el fondo, `ink` es el texto.

Tipografía

- **Baloo 2** (`font-display` , `font-display-bold` , `font-display-medium`) — títulos, montos, nombres de plato. Es la tipografía redondeada y juguetona que le da personalidad a la marca.
- **Nunito Sans** (`font-body` , `font-body-semibold` , `font-body-bold` , `font-body-extrabold`) — todo el texto de lectura: descripciones, labels, botones secundarios.

Mascotas

En `src/assets/mascots/` :

- **Papi Papa** — la papa protagonista, mascota principal de la app. Cuerpo redondeado tipo sticker, guantes blancos, moño rojo. Tiene variantes:
- `papi_papa.png` — versión base, en la elección de modo (onboarding).
- `papi_papa_checklist.png` — caminando, usada en la barra de progreso del wizard (`WizardScreen`), en la pestaña Plan y en recuperar contraseña.
- `papi_papa_saludo.png` — medio cuerpo, saludando, en login y registro.
- `papi_papa_apunta.png` — medio cuerpo, señalando, en la pestaña Plan.

- `papi_papa_costa.png`, `_sierra.png`, `_selva.png` — variantes regionales (con sombrero/vestimenta típica de cada región), para elegir la región en "¿Quién eres?" y en la pantalla "generando tu plan".
- **Choclito** — mascota secundaria (`choclito.png`), para el Flujo B (delivery inmediato): en Explorar y al calificar un pedido.
- **Cuy Delivery** (`cuy_delivery.png`) — mascota del repartidor/delivery: en seguimiento y en pedidos, y en la elección de modo.
- **Billetera** (`billetera.png`) — en la tarjeta de presupuesto de la pestaña Plan.

Los archivos `choclito_ref.jpg` y `papi_papa_ref.jpg` son referencias de arte; la app no los usa.

Logos (`src/assets/images/`)

- `logo_wordmark.png` — logo con el nombre, en login, registro y ubicación.
- `logo_icon.png` — ícono de la app (`app.json` → `icon`).
- `logo_icon_foreground.png` — capa de primer plano del ícono adaptativo de Android (fondo `#FDF3E7`).
- `logo_splash_padded.png` — imagen de la pantalla de carga (`expo-splash-screen`).

Todo el arte de marca (mascotas, íconos de antojos/restricciones/ingredientes, fondos de cocina) es generado por el dueño del producto y entregado como PNG con transparencia — no se dibuja a mano en código. El estilo consistente en todos ellos: contorno grueso oscuro (`#241F21`), colores planos vibrantes, sombreado suave tipo cel-shading, borde blanco tipo sticker.

Estilo visual general

- Tarjetas blancas redondeadas (`rounded-2xl` / `rounded-3xl`) sobre fondo `cream`.
- Sombras suaves (nunca bordes duros) para dar sensación de "objetos apilados".
- Íconos siempre a color, nunca monocromáticos genéricos (excepto la tab bar, que usa PNGs monocromos con `tintColor` para el estado activo/inactivo).
- Tono de la copy: cercano, en español peruano informal ("¿Qué se te antoja esta semana?", "Voy a comprarlo yo"), sin tecnicismos.

CAPÍTULO 7 · DOCS/DATASET.MD

Dataset de recetas

Base de recetas en español para la búsqueda y la lista de compras. Se arma con dos scripts en `scripts/dataset/`; los datos procesados viven en `data/` (ignorado por git, se regenera).

```
pip install pandas
python scripts/dataset/build_recipes.py    # une, limpia y normaliza  -> data/processed/recipes.csv + ingredients.
python scripts/dataset/build_viewer.py     # visor HTML para revisar  -> data/processed/visor/index.html
```

`build_recipes.py` lee los CSV de la carpeta de descargas (`--downloads`, por defecto `~/Downloads`) y escribe en `--out` (por defecto `data/processed`). `build_viewer.py` lee `--data` (por defecto `data/processed`).

Fuentes

Cinco CSV descargados manualmente (no se versionan): `recetasdelaabuela.csv` y `main.csv` (~97 % solapados entre sí), y `train.csv` / `valid.csv` / `test.csv` (esquema de 5 columnas, ingredientes pegados sin separador). Resultado: ~67 mil filas de entrada → ~28.8 mil recetas únicas.

Licencia: las recetas vienen de sitios comerciales (recetasgratis.net, elmueble.com, etc.) y no se conocen los términos bajo los que se publicaron. Sirve para prototipo o curso; **antes de usarlo en un producto real hay que revisarlo**. Por eso no se sube al repo ni se publica el visor.

Qué hace `build_recipes.py`

- **Deduplica** por URL específica (las URLs de portada, p. ej. `https://www.yanuq.com/`, no cuentan como identidad) o por título. Los títulos repetidos casi siempre son recetas distintas; solo ~20 son duplicados reales y se marcan en `posible_duplicado_de`.
- **Repara** texto dañado (`orÃ©gano` → `orégano`), limpia basura de scraping en los pasos ("Calorías 572", "Maridaje") y pasa títulos EN MAYÚSCULAS a formato normal.
- **Ingredientes:** los separa, quita créditos ("Enviada por..."), encabezados ("Para la masa:") y utensilios, y pega modificadores sueltos ("picado", "en cubos") al ingrediente anterior. En `train/valid/test` la separación es heurística (`parseo_heuristico`); lo dudoso lleva `parseo_dudoso`.
- **País:** normaliza las escrituras (`Peru` / `Perú`, `ESP` / `España`) y, si falta, lo infiere por dominio (`pais_inferido`).
- **Dietas:** las etiquetas vegetariana/vegana de la fuente **no se creen**: solo se marca `apta_*` si además ningún ingrediente es animal. Las que la fuente marcó y los ingredientes contradicen quedan en `dieta_en_conflicto`. Esto no es una verificación de alérgenos.

Cantidades

Cada ingrediente tiene la cantidad original y una **normalizada**: `cantidad_base` + `unidad_base` (`g`, `ml`, `unidad` / `diente` / `lata` ..., `pizca`, `al_gusto`). La columna `metodo_cantidad` dice de dónde salió:

Método	Significado
`directa`	gramos, kg, ml, litros: conversión exacta
`conversion_aproximada`	cucharadas (15 ml), cucharaditas (5 ml), tazas (240 ml), vasos (200 ml); también "1 k" = kilo. **No** se convierten a gramos: una taza de harina y una de agua no pesan lo mismo
`equivalencia`	la fuente trae `(400 g)` junto a una lata o unidad
`conteo`	"2 limones", "3 dientes de ajo": se guarda el conteo
`al_gusto` / `al_gusto_supuesto`	"sal al gusto"; el segundo es un supuesto mío para "Sal" sin cantidad
`sin_dato`	la fuente no dice cuánto; **no se inventa**

Valores que la fuente trae mal (p. ej. "400 kilogramos de nueces", "170 Harina" sin unidad) se marcan en `cantidad_sospechosa` en vez de corregirse por adivinanza, y se excluyen de `cantidad_por_racion`. Esta última solo existe si la receta declara comensales (≈ 47 % de las recetas).

Tabla de peso por unidad

`pesos_por_unidad.csv` estima cuánto pesa cada conteo ("1 limón ≈ 60 g", "1 diente de ajo ≈ 4 g", "1 chorro de aceite ≈ 15 ml") para poder sumar cantidades entre recetas y compararlas con lo que se vende por kg.

- **Los valores son estimaciones culinarias estándar, no mediciones.** Cada fila trae mínimo, máximo y `confianza` (alta/media/baja); en la salida quedan como `gramos_estimados`, `gramos_est_min`, `gramos_est_max`, `ml_estimados` y `peso_confianza`.
- **Se guardan aparte de la medida real.** `gramos_totales` usa la medida real cuando existe y la estimada si no; `origen_gramos` (`medida` / `estimada`) dice cuál es.
- **Hay reglas por país** donde el producto cambia: el "limón" de Perú, México, Colombia y otros es una lima pequeña (≈ 30 g, no 60 g), y en varios países sudamericanos el "pimentón" es el pimiento morrón.
- **Confianza baja a propósito** en cosas muy variables (latas de atún, ajíes, pechugas, manojos de hierbas).
- **Cobertura actual:** ≈ 71 % de los conteos tiene peso o volumen estimado; se conoce el peso total de ≈ 48 % de todos los ingredientes (≈ 21 % medida real, ≈ 27 % estimada). Al final de cada corrida, `build_recipes.py` imprime las combinaciones frecuentes **sin regla**, para ir ampliando la tabla.

Visor de revisión

`build_viewer.py` genera un visor local (doble clic en `index.html`, sin servidor): busca por título o ingrediente, filtra por país y fuente, y tiene casillas para las marcas de revisión (parseo dudoso, cantidad sospechosa, duración sospechosa, comensales sospechosos, posible duplicado). Muestra cada ingrediente con su texto original, la cantidad normalizada, el peso estimado y las etiquetas heurístico, dudoso, animal y catálogo. Está pensado para recorrer recetas antes de cargarlas a Supabase. Los datos van en fragmentos `datos/` que se cargan con `<script>`, por eso no necesita servidor.

Pendiente

- Cargar a Supabase (con pgvector para búsqueda) una vez revisado.
- Categorizar cada receta y ampliar el catálogo de ingredientes (hoy solo 12 % de los ingredientes calza con `INGREDIENTES`).
- Ampliar `pesos_por_unidad.csv` con las combinaciones que siguen sin regla.

CAPÍTULO 8 · DOCS/LOCALHOST.MD

Correr Mikhuna en localhost (web) — guía de trabajo

> Cómo levantar la app en el navegador para probar y verificar cambios, y cómo verla con el tamaño de un celular. Para generar el APK de Android ver COMPILACION_APK_ANDROID.md. Para el historial de cambios y cómo regresarlos ver CHANGELOG.md. Para la IA ver IA.md.
>
> Adaptado de la guía equivalente de NovaSmart. Todo lo que dice acá fue comprobado en este repo; lo que no se pudo comprobar está marcado como "no verificado".

0. Lo más importante: hay una sola base de datos y puede estar pausada

Localhost solo levanta la **interfaz**. El inicio de sesión, el registro, los planes, los pedidos y la función de IA viven en **un único proyecto de Supabase**, *mikhuna* (ref `lvkweyiwzkqtojlylinc`, región `sa-east-1`, São Paulo). No existe una copia de pruebas: todo lo que se guarde desde localhost queda en ese proyecto.

Las tablas (`planes`, `pedidos`) se crean con las migraciones de `supabase/migrations/`. En un proyecto nuevo, o después de agregar una migración:

```
npx supabase link --project-ref lvkweyiwzkqtojlylinc   # pide la contraseña de la base de datos
npx supabase db push                                    # aplica las migraciones que falten
```

El plan gratuito de Supabase pausa los proyectos por inactividad. Un proyecto pausado se reconoce así:

Señal	Qué se ve
App	Gira unos segundos al abrir y cae al login; registrar o ingresar falla con "Failed to fetch"
Consola del navegador	`net::ERR_NAME_NOT_RESOLVED` hacia `lvkweyiwzkqtojlylinc.supabase.co`
Terminal	`npx supabase projects list` muestra `"status":"INACTIVE"`
DNS	`nslookup lvkweyiwzkqtojlylinc.supabase.co` responde "Non-existent domain"

Solución: entrar al dashboard de Supabase, abrir el proyecto *Mikhuna* y pulsar **Restore project**. Tarda unos minutos; cuando `projects list` muestre `ACTIVE`, recargar la app. Mientras esté pausado, la función `generar-plan` tampoco responde.

Comprobar que está arriba:

```
npx supabase projects list
```

1. Requisitos (una sola vez)

Qué	Versión comprobada	Nota
Node.js	22.16.0	`node -v`
npm	10.9.2	`npm -v`
Git	cualquiera reciente	

```
cd <carpeta del repo>
npm install
```

Variables de entorno

La app lee sus claves de `.env.local`, que está en `.gitignore` y **no viaja por git**. En otra PC hay que copiarlo a mano (nunca por git ni por chat sin cifrar). Expo lo carga solo: al arrancar la terminal muestra `env: load .env.local`.

```
EXPO_PUBLIC_SUPABASE_URL
EXPO_PUBLIC_SUPABASE_ANON_KEY

# opcionales, solo para el inicio de sesión automático en desarrollo (§5)
EXPO_PUBLIC_DEV_EMAIL
EXPO_PUBLIC_DEV_PASSWORD

# solo para scripts/supabase/aplicar-plantillas-correo.ps1 (la app no las lee)
RESEND_API_KEY        # Resend (recomendado); la clave empieza con re_
SMTP_REMITENTE        # remitente verificado en Resend, p. ej. no-reply@tu-dominio
SMTP_GMAIL_USER       # alternativa: Gmail
SMTP_GMAIL_APP_PASSWORD
```

Solo van la URL y la clave pública (`anon`). La clave de Gemini **no está en el cliente**: es un secreto del proyecto de Supabase (`GEMINI_API_KEY`, ver IA.md). Sin estas dos variables la app abre pero no conecta con Supabase.

Las plantillas de correo (confirmar registro y recuperar contraseña) se suben con:

```
powershell -ExecutionPolicy Bypass -File scripts/supabase/aplicar-plantillas-correo.ps1
```

Ese script usa el token que deja `npx supabase login` y solo cambia el SMTP si `.env.local` trae con qué configurarlo.

---

2. Levantar el servidor web

```
npm run web           # = expo start --web  →  http://localhost:8081
```

La primera vez Metro compila el bundle y puede tardar; cuando `http://localhost:8081` responde, abrirlo. Los cambios de código se recargan solos (hot reload). Para detenerlo: `Ctrl + C`.

Otro puerto (por ejemplo para tener dos sesiones a la vez):

```
npx expo start --web --port 8082
```

Con Claude Code (panel de vista previa)

`.claude/launch.json` define el servidor `mikhuna-web` (puerto 8081); se arranca por nombre desde el panel.

Comprobar que responde:

```
curl.exe -s -o NUL -w "%{http_code}" http://localhost:8081     # 200 = listo
```

---

3. Verlo como celular

En web la app se limita a **430 px de ancho y se centra** en la ventana; los lados quedan en color `ink`. Lo hace `PhoneFrame` en `app/_layout.tsx` y solo corre en web (en el celular no hace nada). En una ventana más angosta que 430 px ocupa todo el ancho. Para ver el tamaño exacto de un modelo de celular, fijar el dispositivo en el navegador:

Dónde	Cómo
Chrome / Edge	`F12` → `Ctrl + Shift + M` (barra de dispositivos) → elegir *iPhone 14 Pro* o escribir `390 × 844`. Recargar la página después de cambiar el dispositivo.
Panel de Claude Code	Menú *Viewport* del panel → *Mobile* (375 × 812). Volver a *Desktop* al terminar.
Celular real	Misma red Wi-Fi: `npx expo start` y abrir el código QR en Expo Go (no verificado en este repo). Si la red bloquea la conexión entre dispositivos (pasó en la red del campus), probar `npx expo start --tunnel` (no verificado).

Tamaños útiles: **375 × 812** (iPhone clásico), **390 × 844** (iPhone 14/15), **360 × 800** (Android común).

Diferencias conocidas entre la web y el celular

- **`SafeAreaView` en web (corregido el 6 oct 2026).** El `SafeAreaView` de `react-native-safe-area-context` emite `padding: 0` explícito en web y pisaba las clases de Tailwind (`px-6` medía `0px` en el login: los campos tocaban los bordes). Ahora todas las pantallas importan `SafeAreaView` desde `components/SafeAreaView`: en el celular es el mismo de la librería; en web (`SafeAreaView.web.tsx`) es un `View` plano que respeta el `className`. **En pantallas nuevas, importarlo desde ahí y no desde la librería.** No se comprobó si el problema original ocurría también en el celular.
- **Ubicación:** la pantalla de ubicación usa `expo-location`; en web el permiso lo pide el navegador (no verificado).
- **Fuentes:** Baloo 2 y Nunito Sans se sirven desde el propio servidor local (`@expo-google-fonts`), no se descargan de internet.

4. Verificar un cambio (flujo de trabajo)

1. Editar el código (el servidor recarga solo).
2. Revisar tipos: `npx tsc --noEmit`. El 21 oct 2026 pasa **sin errores**; si aparece alguno, es nuevo.
3. Si el cambio toca las reglas del plan o las recetas: `npm test` (10 pruebas de `scripts/pruebas/plan.test.ts`).
4. Probar en el navegador **con tamaño de celular** (§3), en el flujo completo que toca el cambio.
5. Cambios en la Edge Function: volver a desplegar con `npx supabase functions deploy generar-plan --no-verify-jwt` (ver IA.md). Cambios en `supabase/migrations/`: `npx supabase db push` (§0).
6. Dejar el registro: una entrada nueva arriba en CHANGELOG.md con "Qué" y "Cómo regresar", y commit.

5. Problemas conocidos de la versión web (y qué hacer)

Síntoma	Causa	Solución
La app gira, cae al login y registrar/ingresar falla	Proyecto de Supabase pausado (§0)	Restaurarlo desde el dashboard
Se ven elementos duplicados o viejos (por ejemplo una mascota donde no va) tras muchos cambios	Pestaña con caché de hot reload	Cerrar la pestaña por completo y abrirla de nuevo
La pantalla "Generando tu plan" no termina de animar en el navegador del panel de Claude Code	La animación no avanza en esa pestaña aunque esté al frente	Probar en un navegador normal o en el celular
Un `Pressable` dentro de otro `Pressable` puede no recibir el clic	Problema conocido de la web (no se reprodujo en esta sesión)	Evitarlo: poner los dos como hermanos dentro de un `View`, como se hizo en `app/(plan)/lista-compras.tsx`
Un `Alert.alert` con botones no hace nada	En web `Alert` no muestra botones	Hoy el código **no lo usa** (verificado con búsqueda). Si se agrega, dibujar un cuadro propio
Un elemento se mueve y arrastra a los demás	Se usó `marginTop` / `marginBottom`	Para mover solo un elemento usar `transform: [{ translateY: n }]`, que no afecta a los vecinos

Inicio de sesión automático en desarrollo

En desarrollo (`__DEV__`), `context/AuthContext.tsx` intenta iniciar sesión solo con una cuenta de prueba para no tener que escribir los datos en cada recarga. Si falla (sin red o proyecto pausado), cae al login normal. En un build de producción no corre. Las credenciales se leen de `EXPO_PUBLIC_DEV_EMAIL` y

`EXPO_PUBLIC_DEV_PASSWORD` en `.env.local`; si no están, no intenta nada y muestra el login normal. Usar una cuenta solo de pruebas, nunca una personal: las variables `EXPO_PUBLIC_*` quedan dentro del bundle de desarrollo.

6. Probar en el emulador de Android (opcional)

```
npm run android        # = expo run:android
```

Necesita Android Studio con un emulador creado y el SDK configurado (`ANDROID_HOME`). Genera un build de **desarrollo** que se conecta al servidor Metro. Para instalar la app "tal cual", generar el APK de release: COMPILACION_APK_ANDROID.md.

7. Referencia rápida

```
npm install                                  # instalar dependencias
npm run web                                  # servidor web en :8081
npx expo start --web --port 8082             # segundo servidor
npx tsc --noEmit                             # revisar tipos
npm test                                     # pruebas de las reglas del plan
npx supabase projects list                   # ¿el proyecto está ACTIVE?
curl.exe -s -o NUL -w "%{http_code}" http://localhost:8081
```

Qué	Dónde
App en el navegador	http://localhost:8081
Repositorio	https://github.com/EdwardPM05/mikhuna
Cambios y cómo regresarlos	CHANGELOG.md

Si algo no se parece a lo descrito acá, revisar el último cambio en CHANGELOG.md.

Compilación del APK Android — Mikhuna (Windows)

> Adaptado de la guía equivalente de NovaSmart (`C:\nova\COMPILACION_APK_ANDROID.md`). Build **siempre local** para este proyecto — no usar EAS Build en la nube salvo que se decida lo contrario explícitamente.

0. El proyecto es "bare workflow"

Desde que se corrió `npx expo prebuild --platform android` , `android/` es un proyecto Android nativo real con Gradle — no se regenera solo. Si se cambia un ícono/splash/nombre en `app.json` , hay que volver a correr `npx expo prebuild --clean` para que se refleje en `android/` .

1. Las 3 formas de compilar, y cuándo usar cada una

Método	Comando	Qué genera	Cuándo usarlo
Debug local	`cd android && ./gradlew assembleDebug`	APK que necesita `npx expo start` corriendo (Metro) para funcionar.	Iterar rápido con hot reload durante desarrollo.
Release local	`cd android && ./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a`	APK standalone, JS empaquetado adentro, no necesita PC prendida.	Probar el build real en el celular, mandarlo a alguien.
~~EAS Build (nube)~~	—	—	No usar para este proyecto por ahora — build siempre local.

2. Antes de compilar (una vez por PC)

Keystore de debug. `android/app/build.gradle` firma también el release con `android/app/debug.keystore` , que no viaja por git (`*.keystore` está en `.gitignore`). Si no existe, copiar el que genera Android Studio:

```
cp ~/.android/debug.keystore android/app/debug.keystore
```

SDK de Android. Gradle necesita `ANDROID_HOME` apuntando al SDK:

```
export ANDROID_HOME="$LOCALAPPDATA/Android/Sdk"
```

3. Reducir el tamaño del APK: arquitecturas de CPU

Por defecto `android/gradle.properties` empaqueta las 4 arquitecturas (`armeabi-v7a`, `arm64-v8a`, `x86`, `x86_64`). Casi cualquier celular real actual es `arm64-v8a`; las otras son para equipos viejos o emuladores. Se puede limitar por build sin tocar ningún archivo, con `-PreactNativeArchitectures`:

```
cd android
./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a
```

4. Comandos de referencia (copiar/pegar)

Release liviano, solo `arm64-v8a`:

```
cd android
./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a --stacktrace
```

El APK queda en `android/app/build/outputs/apk/release/app-release.apk`.

Debug (necesita `npx expo start` corriendo):

```
cd android
./gradlew assembleDebug -PreactNativeArchitectures=arm64-v8a
```

Limpiar cachés nativas si algo se comporta raro:

```
rm -rf android/app/.cxx android/app/build android/build android/.gradle
```

Si es específicamente un asset/imagen desactualizado, limpiar también Metro:

```
rm -rf "$TEMP/metro-cache" "$TEMP"/haste-map-* .expo
```

5. Qué lleva el APK release

- **Sin credenciales de desarrollo.** `EXPO_PUBLIC_DEV_EMAIL` y `EXPO_PUBLIC_DEV_PASSWORD` solo se usan dentro de un `__DEV__ ?` en `context/AuthContext.tsx`, así que el build de release no las incluye. Comprobado: el bundle `assets/index.android.bundle` del APK compilado el 21 oct 2026 (01:56) no contiene los valores de esas variables.
- **Sin respaldo automático de datos.** `allowBackup` está en `false` en `app.json` (`android.allowBackup`) y en `android/app/src/main/AndroidManifest.xml`.
- **Sí lleva** `EXPO_PUBLIC_SUPABASE_URL` y `EXPO_PUBLIC_SUPABASE_ANON_KEY`: son públicas por diseño (la clave `anon`). La clave de Gemini no está en el APK.

6. Instalar sobre un build anterior con distinta firma

Mientras no haya keystore de producción propio, `assembleRelease` firma con `android/app/debug.keystore` (contraseña `android`, alias `androiddebugkey`, definidos en `signingConfigs.debug` de `android/app/build.gradle`). Sirve para probar en el celular, pero **no** para distribuir de verdad — ver §7. Si Android se queja de "conflicto de firma" al instalar sobre una versión vieja, desinstalar el APK anterior primero.

7. Pendiente antes de distribuir de verdad (no bloquea pruebas)

- [ ] Generar un keystore de producción propio para Mikhuna (nunca reusar el de NovaSmart) cuando se vaya a repartir la app fuera de pruebas internas: `bash keytool -genkeypair -v -keystore android/app/mikhuna-release.jks -alias mikhuna -keyalg RSA -keysize 2048 -validity 10000 -dname "CN=Mikhuna, OU=Mikhuna, O=Mikhuna, L=Lima, S=Lima, C=PE"`
- Guardar `android/app/mikhuna-release.jks` + `android/keystore.properties` fuera de esta PC (backup). Si se pierde, no hay forma de publicar updates sobre una versión ya instalada con esa firma.
- [ ] Configurar `signingConfigs.release` en `android/app/build.gradle` para leer ese keystore (hoy `buildTypes.release` usa `signingConfigs.debug`). Igual patrón que NovaSmart §10 de su guía.
- [ ] Considerar fijar `reactNativeArchitectures=arm64-v8a` en `android/gradle.properties` si todos los builds van a ser solo para celulares modernos (hoy trae las 4).

CAPÍTULO 10 · DOCS/CHANGELOG.MD

Registro de cambios — Mikhuna

Cada cambio medianamente importante (pantalla, flujo, función de IA, dataset o documentación) queda aquí con su commit, lo que tocó fuera del código y **cómo regresarlo**. Lo más reciente va arriba. El hash permite ver el detalle con `git show <hash>`. Cómo correr el proyecto: LOCALHOST.md.

Cómo regresar un cambio

- **Código:** `git revert <commit>` (deja un commit nuevo que lo deshace; no borra historia). Si el cambio tiene varios commits, revertir del más nuevo al más viejo.
- **Edge Function (`generar-plan`):** vive en el servidor, no en la app. Después de revertir el archivo hay que volver a desplegarla: `npx supabase functions deploy generar-plan --no-verify-jwt`. Para quitarla del servidor: `npx supabase functions delete generar-plan`. Su clave es un secreto del proyecto (`npx supabase secrets set/unset GEMINI_API_KEY`).
- **Base de datos:** las tablas se crean con las migraciones de `supabase/migrations/` (`npx supabase db push`). Para deshacer una, se escribe otra migración que la revierta; no se edita una que ya se aplicó. Hay **un único proyecto de Supabase**, *mikhuna* (ref `lvkweyiwzkqtojlylinc`, `sa-east-1`), que sirve a desarrollo y a producción: no hay copia de pruebas.
- **Dataset:** `data/` está en `.gitignore` y se regenera. Tras revertir un commit de `scripts/dataset/`, volver a correr `python scripts/dataset/build_recipes.py` (y `build_viewer.py` si se usa el visor).

---

21 oct 2026

Plan según quién eres, con IA que elige entre recetas reales

- **Qué:** paso nuevo "¿Quién eres?" al inicio del wizard; el plan ahora respeta restricciones, electrodomésticos y presupuesto (antes un vegano que pedía criolla recibía lomo saltado); 12 recetas nuevas (sierra, selva, veganas, sin gluten, con horno, de otras cocinas); Gemini elige entre esas recetas y explica por qué, con las reglas de respaldo. Las recetas y las reglas viven en `supabase/functions/_shared/` y las usan la app y la función. `npm test` corre las pruebas de las reglas. Diseño en ENFOQUE.md, detalle en IA.md.
- **Fuera del código:** volver a desplegar la función (`npx supabase functions deploy generar-plan --no-verify-jwt`). Mientras no se despliegue, la app arma el plan con las reglas.
- **Cómo regresar:** `git revert` de los commits (del más nuevo al más viejo) y volver a desplegar la función. Los planes guardados con la versión nueva traen campos de más (`perfil`, `planSemana`…) que la versión vieja ignora.

Revisión de seguridad y de errores

Se revisó toda la app y la parte de Supabase. Cada arreglo tiene su commit (`git log --since=2026-10-21`). - IA (`generar-plan`): exige un usuario con sesión, solo acepta valores que la app puede mandar, limita los tokens de salida, manda la clave de Gemini en un header y no devuelve el detalle de los errores. **Fuera del**

código: hay que volver a desplegarla (`npx supabase functions deploy generar-plan --no-verify-jwt`). **Cómo regresar:** `git revert` del commit y volver a desplegar. - **Base de datos:** `20261021030000_limites_de_tamano.sql` pone topes de largo a los textos de `pedidos` y de tamaño a `items` y `planes.datos`. **Fuera del código:** `npx supabase db push`. **Cómo regresar:** una migración nueva con `alter table ... drop constraint` de cada una. - **APK:** el correo y la contraseña de la cuenta de pruebas (`EXPO_PUBLIC_DEV_*`) quedaban dentro del APK release; ya no. El backup de Android quedó desactivado (`allowBackup: false`) porque copiaba la sesión fuera del celular. Cualquier APK compilado antes de este cambio trae esa contraseña: no compartirlo y cambiar la contraseña de la cuenta. - **Pedidos:** una carga vieja ya no pisa a una nueva; si el servidor rechaza un cambio de estado o una calificación se deshace en pantalla; los pedidos guardados en el celular ya no se suben dos veces; "Volver a pedir" usa el precio de hoy; el checkout vuelve a exigir el pedido mínimo. - **Sesión:** si la sesión se pierde fuera del login la app vuelve al login, y el carrito se vacía al cambiar de cuenta. - **Compras del plan:** "Pedir de X" ya no duplica cantidades ni mezcla tiendas, y no pide lo que ya se marcó como comprado. - **Ubicación:** "Usar mi ubicación actual" detecta el distrito con el GPS (antes llevaba a un mapa de mentira que no guardaba nada) y la lista muestra los 30 distritos. - **Accesibilidad:** los botones de solo ícono y `BrandButton` se anuncian bien con TalkBack/VoiceOver.

---

20 oct 2026

Correos de registro y recuperación con diseño

- **Qué:** `supabase/templates/confirmar-registro.html` y `recuperar-contrasena.html`: fondo crema, logo, Papi Papa, saludo con el nombre del registro y el código en una caja grande punteada. Asuntos: "Tu código para entrar a Mikhuna 🥔" y "Tu código para cambiar la contraseña 🔑". Las imágenes se cargan desde GitHub (el repo es público).
- **Cómo se aplican:** `scripts/supabase/aplicar-plantillas-correo.ps1` las sube con la API de administración de Supabase y solo cambia esos cuatro campos de Auth.
- **Cómo regresar:** `git revert` del commit y, en el dashboard (Authentication → Emails), "Reset to default" en cada plantilla.

Registro confirmado con código

- **Qué:** después de registrarse se escribe el código del correo y se entra directo (antes llegaba un enlace que no abría la app y la cuenta quedaba sin confirmar). El login ofrece "Confirmar con un código" a cuentas sin confirmar. También se corrigió la traducción de errores: un código vencido o mal escrito mostraba "Ese correo no parece válido".
- **Fuera del código:** la plantilla *Confirm signup* de Supabase tiene que incluir `{{ .Token }}`.
- **Cómo regresar:** `git revert` de los commits.

Olvidé mi contraseña

- **Qué:** enlace "¿Olvidaste tu contraseña?" en el login y pantalla nueva `app/(auth)/recuperar.tsx`: correo → código por correo → contraseña nueva → entra a la app. Funciones `enviarCodigoRecuperacion` y `restablecerContrasena` en `AuthContext`. Detalle y plantilla de correo en FLUJOS.md.

- **Fuera del código:** hay que cambiar la plantilla *Reset Password* de Supabase para que incluya `{{ .Token }}`; con la plantilla por defecto el correo trae un enlace y no un código. Sin SMTP propio solo llegan correos al equipo del proyecto.
- **Cómo regresar:** `git revert` del commit (y, si se quiere, volver a la plantilla por defecto en Supabase).

Planes y pedidos en Supabase

- **Qué:** nueva migración `supabase/migrations/20261021020000_planes_y_pedidos.sql` con las tablas `planes` y `pedidos` y RLS (cada usuario solo ve y cambia lo suyo; de un pedido solo se pueden cambiar estado, calificación y comentario). `OrdersContext` y `PlanWizardContext` ahora leen y escriben ahí. Lo que hubiera en AsyncStorage se sube una vez.
- **Hay que aplicarla:** `npx supabase db push` (con el proyecto vinculado). Hasta entonces la pestaña Pedidos muestra "Sin conexión", confirmar un pedido falla con aviso y el plan sigue guardándose solo en el celular.
- **Docs:** `LOCALHOST.md`, `IA.md` y este archivo apuntaban al proyecto viejo `lpnujzuatgnekirfdxru`, que ya no existe; ahora apuntan a *mikhuna* (`lvkweyiwzkqtojlylinc`).
- **Cómo regresar:** `git revert` de los commits; para quitar las tablas, una migración nueva que las elimine (borra los datos).

Ronda de mejoras de UI y flujo (11 commits, `0e3be8b` … `aeddfbc`)

Todo lo de abajo se comprobó con `npx tsc --noEmit` y recorriendo la app en web (Chrome, 390×844): login, registro, Explorar, negocio → carrito → checkout → seguimiento → Pedidos, Perfil y recarga con el plan guardado. - **Saludo real** (`0e3be8b`): el home decía "Hola, Edward" a todos; ahora usa el nombre del registro. - **Pedidos de verdad** (`b5451f1`): nuevo `context/OrdersContext.tsx`. Confirmar en checkout guarda el pedido; el seguimiento muestra el negocio real (antes siempre "Bodega Don José"), avanza el estado guardado, tiene botón de cerrar y resumen; la calificación queda en el pedido. - **Pestaña Pedidos** (`d248791`): historial en curso / anteriores, "Calificar" y "Volver a pedir". - **Perfil** (`448d90b`): datos de la cuenta, accesos y **cerrar sesión** (antes no había forma de salir). - **Checkout** (`79dca26`): resumen del pedido, desglose subtotal + propina y aviso cuando falta la dirección. - **Explorar** (`367d5de`): buscador (nombre, tipo y productos), estado vacío y barra flotante del carrito. - **Pestañas** (`ccb0e15`, `aeddfbc`): contador de pedidos en curso, fuente de marca y etiquetas que ya no se cortan en web. - **Plan guardado** (`559fd7d`): el plan semanal sobrevive a recargar la app (antes se perdía). - **Entrada directa** (`8b7aca0`): quien ya eligió distrito entra a la app sin repetir el onboarding. - **Login y registro** (`7893a60`, `e703ff1`, `bac4d4d`): campos con ícono y ver/ocultar contraseña, errores en español, teclado que no tapa los campos, registro que entra directo si Supabase no pide confirmar el correo, y el auto-login de desarrollo ya no deja al usuario trabado en el login. - **Ojo:** pedidos y plan viven en el dispositivo (AsyncStorage), no en Supabase: otro celular no los ve. Pasarlos a tablas es el siguiente paso natural. - **Cómo regresar:** `git revert` de cada commit, del más nuevo al más viejo. Lo guardado en AsyncStorage (`mikhuna:plan:*`, `mikhuna:pedidos:*`) queda en el dispositivo, pero nada lo lee después del revert.

Aviso de Expo `userInterfaceStyle` silenciado

- **Qué:** `app.json` define `"userInterfaceStyle": "light"` también a nivel `expo` (antes solo en `ios` y `android`). Es el mismo valor, así que la app se ve igual; solo desaparece el aviso de la terminal. En `android/` (bare workflow) no cambia nada hasta correr `npx expo prebuild`.
- **Cómo regresar:** `git revert` del commit.

Credenciales de prueba fuera del código

- **Qué:** `context/AuthContext.tsx` ya no tiene un correo y una contraseña escritos: `DEV_AUTO_LOGIN` lee `EXPO_PUBLIC_DEV_EMAIL` y `EXPO_PUBLIC_DEV_PASSWORD` de `.env.local`. Sin esas variables no intenta el auto-login y muestra el login normal. `README.md`, `LOCALHOST.md` y `ARQUITECTURA.md` explican las variables.
- **Ojo:** la contraseña anterior sigue en el historial de git (repo público); hay que cambiarla en Supabase.
- Comprobado: `npx tsc --noEmit` sin errores.
- **Cómo regresar:** `git revert` del commit.

Carrito: ya no se pierden ingredientes al pedir un grupo

- **Qué:** `agregar()` en `context/CartContext.tsx` comparaba contra el `negocioId` del último render. `pedirGrupo()` (carrito de ingredientes y comprar por delivery) llama a `agregar()` en un bucle; si el carrito tenía algo de otra tienda, **cada** llamada lo vaciaba y quedaba solo el último ingrediente. Además `reemplazado` se asignaba dentro del updater de `setItems`, que corre después de devolverlo, así que el aviso "tu carrito era de otro negocio" de `negocio/[id].tsx` podía no salir nunca. Ahora el estado del carrito es un solo objeto con una copia síncrona (`useRef`), y la API de `useCart()` no cambia.
- Comprobado: `npx tsc --noEmit` sin errores. **No comprobado en el navegador** (no hay tests en el repo todavía).
- **Cómo regresar:** `git revert` del commit (solo toca `CartContext.tsx`).

Modelo de Gemini configurable (`GEMINI_MODEL`)

- **Qué:** `generar-plan` lee el modelo del secreto `GEMINI_MODEL`; si no está, sigue usando `gemini-2.5-flash` (sin cambio para el proyecto actual). Motivo: con una clave de Google AI Studio nueva, `gemini-2.5-flash` responde `404` ("no longer available to new users"). Probado en un proyecto de Supabase nuevo con `GEMINI_MODEL=gemini-3.8-flash`: devolvió un plan válido con `ingredienteId` del catálogo.
- **Cómo regresar:** `git revert` del commit y volver a desplegar la función; `npx supabase secrets unset GEMINI_MODEL`.

---

6 oct 2026

Web: padding lateral corregido y app limitada al ancho de un celular

- **Qué:**

- Nuevo `components/SafeAreaView.tsx` (en el celular es el de la librería, sin cambios) y `components/SafeAreaView.web.tsx` (en web es un `View` plano). Las 21 pantallas y componentes que usaban `SafeAreaView` ahora lo importan de ahí; es una sola línea por archivo. Causa: en web el de la librería emite `padding: 0` explícito y pisaba clases como `px-6`.
- Nuevo `PhoneFrame` en `app/_layout.tsx`: solo en web, limita la app a 430 px de ancho, la centra, deja los lados en `ink` y recorta lo que se salga (por ejemplo las transiciones entre pantallas). En el celular no hace nada.
- Comprobado en el navegador: a 1024 px de ventana el login queda en una columna de 430 px, con padding de 24 px (antes 0 px) y campos de 382 px (antes 1024 px). Registro, modo, mapa, perfil, pedidos y explorar también miden 24 px; la pantalla de receta (usa `edges`) no da avisos en consola; Plan se ve bien; `npx tsc --noEmit` sin errores.
- **No comprobado:** el wizard completo y la pantalla "Generando" (Supabase pausado y animación que no avanza en el panel), ni el comportamiento en el celular (no debería cambiar: reexporta la librería).
- **Cómo regresar:** `git revert` del commit. A mano: borrar los dos `SafeAreaView*.tsx`, volver a poner `import { SafeAreaView } from "react-native-safe-area-context";` en las 21 pantallas y quitar `PhoneFrame` de `app/_layout.tsx`.

Guía de localhost y changelog en formato por fecha (solo documentación)

- **Qué:** nuevo `LOCALHOST.md` (cómo correr la app en web, cómo verla con tamaño de celular, problemas conocidos). Este registro pasa a organizarse por fecha, con "Qué / Cómo regresar" en cada entrada. El `README.md` enlaza la guía. No se tocó código de la app.
- **Estado comprobado ese día:**
- Git limpio y sincronizado con `origin/main`; `npx tsc --noEmit` pasa con 0 errores; el servidor web levanta en `:8081`.
- El proyecto de Supabase figuraba `INACTIVE` (pausado por inactividad del plan gratuito). Con el proyecto pausado, la app gira unos segundos y cae al login, no se puede registrar ni ingresar, y `generar-plan` no responde. Se restaura desde el dashboard (*Restore project*). Detalle en LOCALHOST.md §0.
- **Hallazgos de esa revisión:** padding lateral perdido en web (corregido en la entrada de arriba), credenciales de prueba en el código y aviso de `userInterfaceStyle` (estos dos quedan en *Pendiente conocido*).
- **Cómo regresar:** `git revert` del commit (vuelven el `CHANGELOG.md` y el `README.md` anteriores y desaparece `LOCALHOST.md`). No hay cambios de base de datos.

---

19 sep 2026 — Documentación, IA y dataset de recetas

Documentación del backend y del dataset (`b0b5d4b`)

- **Qué:** primer `CHANGELOG.md`; `IA.md` y `ARQUITECTURA.md` actualizados (reintentos de Gemini, sección de scripts de datos); el `README.md` enlaza `IA.md`, `DATASET.md` y el changelog.
- **Cómo regresar:** `git revert b0b5d4b` (solo documentación).

Tabla de peso por unidad (`bab8317`)

- **Qué:** nueva `pesos_por_unidad.csv` (~90 reglas) que estima gramos o ml de los conteos ("2 limones" ≈ 120 g) con mínimo, máximo y confianza. Cobertura: 71.6 % de los conteos; peso total conocido en 47.9 % de los ingredientes (20.9 % medido, 27.1 % estimado).
- Reglas por país donde el producto cambia (limón = lima pequeña en Perú, México y Colombia; "pimentón" = pimiento en varios países sudamericanos).
- Las estimaciones se guardan **aparte** de la medida real (`origen_gramos`).
- `k` se interpreta como kilo ("1 k (2 lb)") con salvaguarda: "500 k" solo puede ser gramos.
- Nuevas unidades reconocidas: pieza, rebanada, lámina, bote, tarro, botella, barra, copa, pellizco, ramito.
- Corrección: la regla del ají no calzaba con el singular; el emoji 🧄 salía dañado (mojibake de 4 bytes).
- Nuevo `DATASET.md` .
- **Cómo regresar:** `git revert bab8317` y volver a correr `build_recipes.py` .

Cantidades normalizadas (`4786f1f`)

- **Qué:** todo se lleva a `g` , `ml` , conteo o `al_gusto` , con `metodo_cantidad` para saber si es directa, aproximada, por equivalencia `(400 g)` , conteo o supuesto.
- Cucharadas/tazas/vasos se convierten a **ml** (no a gramos: una taza de harina y una de agua no pesan igual).
- Cantidades que la fuente trae mal ("400 kilogramos de nueces") se marcan como sospechosas (1,302), no se corrigen.
- `cantidad_por_racion` solo cuando la receta declara comensales.
- El visor muestra cantidad normalizada, método y por persona.
- **Cómo regresar:** `git revert 4786f1f` y regenerar `data/` .

Filtro de líneas que no son ingredientes (`be45ca4`)

- **Qué:** elimina créditos ("Enviada por..."), encabezados de sección ("Para la masa:", "Ingredientes secos") y utensilios ("Palillos", "Rallador"). Los fragmentos tipo "para freír" se pegan al ingrediente anterior.
- Salvaguarda: "Para la masa 200 g de harina" (con número) se conserva.
- **Cómo regresar:** `git revert be45ca4` y regenerar `data/` .

Visor HTML local (`29e420f`)

- **Qué:** `build_viewer.py` + `visor_template.html` : visor sin servidor con búsqueda por título o ingrediente, filtros de calidad y detalle por receta. Datos en fragmentos de 500 recetas.
- Detectó un defecto que los conteos no mostraban: modificadores ("cocidos", "picado") como ingredientes sueltos; ahora se fusionan.
- **Cómo regresar:** `git revert 29e420f` (el visor se genera en `data/` , no se versiona).

Endurecimiento tras una auditoría completa (`ee75a20`)

- **Qué:** parser de ingredientes reescrito (fracciones, rangos "1 o 2", paréntesis, conectores).
- Texto dañado (`orÃ©gano`) reparado en 620 recetas de ingredientes y 704 de pasos.

- Basura de scraping en pasos ("Calorías 572", "Maridaje") eliminada; títulos en MAYÚSCULAS normalizados (836 → 1).
- **Dietas verificadas contra ingredientes:** la fuente marcaba 1,917 vegetarianas y 1,259 veganas; solo 1,841 y 1,153 se sostienen. Las contradicciones quedan en `dieta_en_conflicto`.
- Banderas de revisión: ingredientes incompletos, duración/comensales sospechosos, posibles duplicados.
- **Cómo regresar:** `git revert ee75a20` y regenerar `data/`.

Reintentos ante 503/429 en la Edge Function (`23c279c`)

- **Qué:** `generar-plan` reintenta hasta 3 veces con espera creciente (0.8 s, 1.6 s, 3.2 s) cuando Gemini está sobrecargado (503) o sin cuota (429). Ver IA.md.
- **Cómo regresar:** `git revert 23c279c` y volver a desplegar la función (ver "Cómo regresar un cambio").

Constructor del dataset (`162bede`)

- **Qué:** `build_recipes.py` une 5 CSV (~67 mil filas) en ~28.8 mil recetas únicas con `recipes.csv` e `ingredients.csv`.
- Corrección de un bug propio: las URLs de portada (`https://www.yanuq.com/`) fusionaban recetas distintas y se perdían ~800 peruanas (Perú: 1,166 → 1,955).
- `data/` y `__pycache__/` pasan al `.gitignore`.
- **Cómo regresar:** `git revert 162bede` (los CSV de `data/` no están en git; se regeneran con el script).

---

17 sep 2026 — Edge Function de IA y commit inicial

Edge Function `generar-plan` con Gemini (`e85f744`)

- **Qué:** función desplegada en Supabase que pide a Gemini un plato por día en JSON estructurado (`responseSchema`). La API key es un secreto del proyecto, nunca va en la app.
- El precio **nunca** lo decide la IA: `lib/ia.ts` lo estima cruzando con `INGREDIENTES`.
- Corrección: Gemini devolvía el nombre del ingrediente en vez de su id; se restringió con un `enum` en el schema.
- `tsconfig.json` excluye `supabase/functions` (Deno).
- Aún **no está conectada al wizard**.
- **Cómo regresar:** `git revert e85f744`; para quitarla del servidor `npx supabase functions delete generar-plan` y `npx supabase secrets unset GEMINI_API_KEY`.

Commit inicial (`ae6355d`)

- **Qué:** estado de la app al subirla a GitHub, con su documentación (`README.md`, `BRANDING.md`, `ARQUITECTURA.md`, `FLUJOS.md`). Incluye:
- Onboarding, autenticación y los dos flujos (plan semanal y delivery inmediato).
- Lista de compras multi-tienda con comparación de precios por ingrediente y carrito por tienda.

- Pantalla **Plan** rediseñada: saludo con Papi Papa, billetera con presupuesto (barra verde/naranja/roja), misión del día y semana de platos.
- **Detalle de receta** (`app/(plan)/receta/[id].tsx`): ingredientes con estado "Ya comprado" cruzado con la lista de compras, pasos numerados y foto real del plato cuando existe (por ahora solo Causa limeña).
- Arte de marca integrado: mascotas (saludo, apunta, regionales), billetera, íconos de antojos/restricciones/ingredientes.
- Modal flotante `transparentModal` para elegir cómo conseguir los ingredientes.
- **Cómo regresar:** es el punto de partida del repo, no se revierte. Para ver cómo estaba, abrirlo en otra carpeta: `git worktree add ../mikhuna-inicial ae6355d` .

Pendiente conocido

- **Reactivar el proyecto de Supabase** si figura `INACTIVE` (ver LOCALHOST.md §0).
- **Cambiar la contraseña de la cuenta que estaba en `AuthContext.tsx` :** ya no está en el código (ver 20 oct 2026), pero sigue en el historial público de git.
- Cargar el dataset a Supabase (con pgvector) tras revisarlo.
- Conectar `generarPlanConIA()` al wizard, con caché y respaldo.
- Categorizar recetas y ampliar el catálogo de ingredientes (solo ~12 % de los ingredientes del dataset calza con `INGREDIENTES`).
- Ampliar `pesos_por_unidad.csv` (28 % de los conteos sigue sin regla) y revisar sus valores contra mercados reales.